

# DynaX: Sparse Attention Acceleration with Dynamic X:M Fine-Grained Structured Pruning

Xiao Xiong  
College of Computer Science,  
Chongqing University  
Chongqing, China  
xiaoxiong@stu.cqu.edu.cn

Zhaorui Chen  
College of Computer Science,  
Chongqing University  
Chongqing, China

Yue Liang  
College of Computer Science,  
Chongqing University  
Chongqing, China

Minghao Tian  
College of Computer Science,  
Chongqing University  
Chongqing, China

Jiaxing Shang  
College of Computer Science,  
Chongqing University  
Chongqing, China

Jiang Zhong  
College of Computer Science,  
Chongqing University  
Chongqing, China

Dajiang Liu  
College of Computer Science,  
Chongqing University  
Chongqing, China  
liudj@cqu.edu.cn

## Abstract

Owning to the mechanism of self-attention, Transformers have exhibited incredible performance in a wide range of artificial intelligence tasks. With the growth of sequence length, attention computation with quadratic complexity becomes the bottleneck, and dynamic sparsity is an effective technique to alleviate this problem. However, dynamic attention sparsity for long-sequence tasks suffers from two challenges, i.e., irregular sparse patterns and heavy prediction overhead. To this end, this paper proposes DynaX, an algorithm-hardware co-design framework that accelerates attention computation via dynamic X:M fine-grained structured pruning. Different from traditional N:M pruning, DynaX dynamically selects variable  $X$  (rather than a fixed  $N$ ) important scores from a group via a 2-step pruning method, which results in high sparsity and less prediction memory overhead while maintaining pattern regularity to a certain extent. After that, DynaX performs block scheduling to reorganize score blocks into hardware blocks that can perfectly match the size of the processing element array (PEA), resulting in a higher utilization rate. Experimental results show that DynaX can achieve

average sparsity of 89.54% and 91.77% for short-sequence tasks and long-sequence tasks, respectively, with less than 1% accuracy loss. Compared to Sanger and SALO2, DynaX achieves a speedup of  $1.99 \times$  and  $1.50 \times$  on the BERT-base model, and an energy efficiency improvement of  $5.16 \times$  and  $4.20 \times$ , respectively.

**CCS Concepts:** • Computer systems organization → Architectures; • Computing methodologies → Machine learning algorithms.

**Keywords:** Self-Attention, Dynamic Sparsity, Acceleration.

## ACM Reference Format:

Xiao Xiong, Zhaorui Chen, Yue Liang, Minghao Tian, Jiaxing Shang, Jiang Zhong, and Dajiang Liu. 2025. DynaX: Sparse Attention Acceleration with Dynamic X:M Fine-Grained Structured Pruning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25)*, March 30-April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3676641.3715991>

## 1 Introduction

Attention-based Transformers [48] have shown exceptional performance in a wide range of artificial intelligence (AI) works, such as natural language processing (NLP) [1, 2, 11, 14, 22, 30, 43, 56], computer vision [5, 10, 27], multimodal tasks [20, 46, 51], etc. In the area of NLP, large language models (LLMs), such as Bloom [22], LLaMA3 [14] and GPT-4 [1] have exhibited incredible performance in various tasks, such as question answering [29], text generation [59], logic reasoning assistance, etc.

The outstanding performance of emerging LLMs mainly comes from the *self-attention* mechanism, which allows the

---

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).  
ASPLOS '25, March 30-April 3, 2025, Rotterdam, Netherlands  
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1079-7/25/03.

<https://doi.org/10.1145/3676641.3715991>

model to capture global context information from the entire sequences. The self-attention block involves 2 matrix multiplication operations on 3 matrices, i.e., Query ( $Q$ ), Key ( $K$ ) and Value ( $V$ ). By multiplying  $Q$  and  $K$ , the self-attention first obtains a score matrix ( $S$ ), which further goes through softmax function for normalization. Then, the normalized score matrix is multiplied with  $V$  to generate the output. Thus, the self-attention is of quadratic complexity, and suffers from heavy computation overhead, especially for tasks with a long input sequence. For example, when handling long-text tasks with bidirectional encoder representations from transformers (BERT) or Vision Transformer (ViT), the attention blocks constitute 47.1%-93.4% of the total computation [36]. Therefore, self-attention blocks are recognized as the bottleneck of Transformers with long sequences, leading to deployment difficulties on end devices.

To address the heavy cost of self-attention when scaling to long sequences, sparse attention is a widely used technique in accelerators [9, 15–18, 23, 24, 28, 36, 37, 40, 50, 57, 58, 61, 62]. Instead of computing the entire score matrix from dense  $Q$  and  $K$ , partial  $Q$  and  $K$  pairs are selected to perform sampled dense-dense matrix multiplication (SDDMM), resulting in a sparse score matrix. Then, sparse-dense matrix multiplication (SpMM) is performed by multiplying the normalized sparse score matrix to  $V$ . Existing sparse attention mechanisms mainly include static sparsity and dynamic sparsity. By pre-determining the sparse pattern, static sparsity [3, 7, 13, 19, 27, 34, 39, 60] ignores the difference among different sequences, leading to limited computation saving at the same accuracy level. Instead, dynamic sparsity dynamically generates the sparse pattern according to the input Queries and Keys, resulting in a better trade-off between accuracy and computation saving. Therefore, our work also focuses on dynamic sparsity.

However, dynamic sparsity also suffers from two challenges when adopting it to hardware accelerators. Firstly, dynamic sparsity may learn arbitrary sparse patterns that are usually highly unstructured. With little regularity, it can incur workload imbalance and random memory access in SDDMM and SpMM operations, leading to a low utilization rate of hardware resources. Secondly, when scaling to long sequences in LLMs, it will inevitably incur non-negligible overheads of predicting the sparse pattern, including both prediction hardware overhead (e.g., computation and memory resources) and prediction latency overhead. Thus, achieving a good balance between accuracy, sparsity and hardware friendliness is a challenging work for sparse attention.

To this end, this paper proposes DynaX, an algorithm-hardware co-design framework that accelerates sparse attention via X:M fine-grained structured pruning. Unlike the traditional N:M pruning [6], DynaX can dynamically select a variable number of scores ( $X$ ), rather than a fixed number of  $N$ , from  $M$  consecutive scores, achieving high sparsity while keeping pattern regularity to a certain extent. From

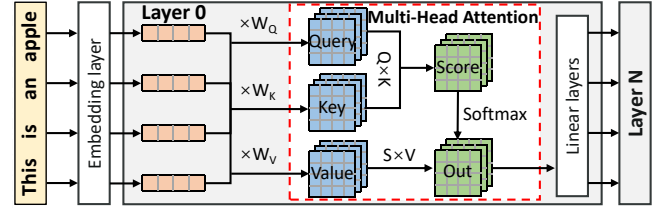


Figure 1. Typical transformer architecture.

the perspective of algorithm, we first pre-compute the score matrix via low-bit  $Q$  and  $K$ . Then, we adopt a 2-step X:M pruning method, including N:M pruning and X:N pruning, for sparse attention. To improve the regularity of sparse pattern, we propose a block scheduling algorithm to reorganize score blocks into *hardware block* that can perfectly match the size of processing element array (PEA) for SDDMM and SpMM. From the perspective of hardware, we elaborate several modules that can efficiently implement each step of the algorithm optimization. The contribution of this paper can be summarized as follows.

1. We propose DynaX, an algorithm-hardware co-design framework that can exploit dynamic fine-grained structured sparsity in attention computation via an efficient and flexible hardware architecture. The framework is open-source and available from <https://github.com/coralabo/DynaX>.
2. We propose a dynamic X:M fine-grained structured pruning method that can enjoy partial regularity of N:M pruning while maintaining high sparsity.
3. We propose an efficient hardware architecture that can not only support the sparsity prediction for long-sequence tasks with significantly reduced memory but also schedule the execution order of score blocks to make good use of hardware resources.

Experiments show that DynaX can achieve average sparsity of 89.54% and 91.77% for short-sequence tasks and long-sequence tasks, respectively, with less than 1% accuracy loss compared to dense attention. Compared to Sanger and SALO2, DynaX achieves a speedup of  $1.99 \times$  and  $1.50 \times$  on the BERT-base model, and an energy efficiency improvement of  $5.16 \times$  and  $4.20 \times$ , respectively. Compared to the V100 GPU, DynaX achieves an average speedup of  $35.14 \times$  and an energy efficiency improvement of  $299.23 \times$ , and DynaX achieved an average energy efficiency of 25.55 TOPS/W for attention computation in all tasks.

## 2 Background

### 2.1 Attention Mechanism in Transformers

Traditional Transformers typically consist of multiple Encoder blocks and Decoder blocks. However, most LLMs [1, 14, 22] nowadays use a Decoder-only architecture, which only includes Decoder layers. Regardless of the Transformer

structure, the attention mechanism is the core module. Figure 1 describes the workflow of the Transformer. Initially, the input token sequence is encoded by the embedding layer and multiplied by the weight matrices  $W_Q$ ,  $W_K$ , and  $W_V$ , followed by splitting, to obtain the multi-head  $Q/K/V$  matrices. The size of each  $Q/K/V$  matrix is  $[W, D]$ , where  $W$  is the length of the input token sequence, and  $D$  is the dimension of a single attention head. Then, the single-head  $Q$  is multiplied by  $K^T$  ( $K$  represents  $K^T$  in the following text) to obtain the Score ( $S$ ) matrix, with a size of  $[W, W]$ , where a larger  $S$  indicates a stronger correlation between the corresponding positions of the two tokens. For the Score matrix, the Decoder needs to set a sequence mask to turn it into a lower triangular matrix. After  $S$  is normalized through softmax, it is multiplied by  $V$  to obtain the output of the attention part, which is then passed through the Linear layer and into the next layer. With multiple matrix-multiplication operations, attention computation is of quadratic complexity and becomes the bottleneck of Transformer when handling long sequences.

## 2.2 Taxonomy of Dynamic Sparse Attention Accelerators

Accelerators exploiting dynamic sparsity can adjust the sparse pattern at inference time based on the input queries and keys, resulting in higher sparsity at the same level of accuracy, as compared to static sparsity. As depicted in Figure 2, we categorize existing or potential accelerators exploiting dynamic sparsity in two dimensions. 1) Sorting/threshold-based selection: are the non-zeros selected by sorting or threshold? 2) Row/block/group-wise pruning: in which scope to conduct the dynamic pruning?

**Row-wise sorting-based pruning.** Accelerators that adopt row-wise sorting-based attention conduct dynamic sparsification in the scope of the entire row of the attention matrix by row-wise sorting. For example, SpAtten [50] performs top- $k$  selection for each row to achieve local value pruning, resulting in considerable sparsity. Since top- $k$  selection involves multiple rounds of sorting on the entire row, it has a high time complexity and is difficult to scale to longer sequences. With the same number of non-zeros per row, it can keep a regular sparse pattern. However, when scaling this category to multiple rows for row-wise parallelization, it will inevitably suffer from heavy memory overhead for prediction since the sorting process needs to hold global information from the entire rows.

**Row-wise threshold-based pruning.** Accelerators that adopt row-wise threshold-based pruning dynamically select non-zeros from the entire row by comparing them to a threshold. For example, we infer that Sanger [28] performs threshold-based pruning in a row-by-row fashion. With just one round of comparing, this kind of accelerator has a low prediction complexity. However, as attention scores in different rows

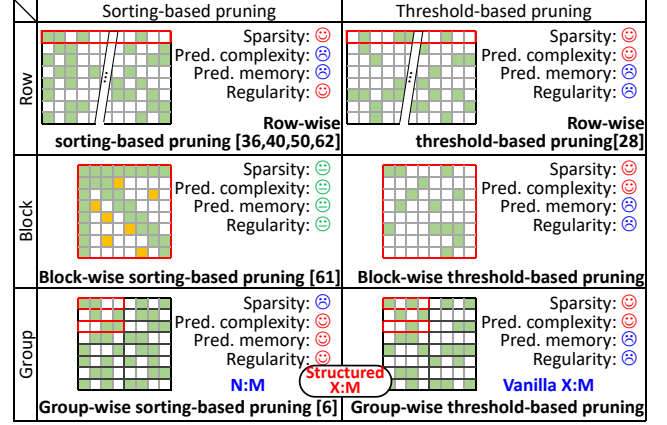


Figure 2. Taxonomy of self-attention accelerators exploiting dynamic sparsity.

share the same threshold, it needs softmax operation to normalize the scores into probabilities for comparison, which involves holding the entire row of scores in on-chip memory. When handling multiple rows simultaneously, this kind of pruning suffers from heavy memory overhead for prediction. Moreover, threshold-based pruning can generate arbitrary sparse patterns for high sparsity yet it also incurs poor regularity.

**Block-wise sorting-based pruning.** Accelerators using block-wise sorting-based attention perform value pruning within a block of attention matrix via value sorting. For example, SALO2 [61] can dynamically generate a hybrid (sliding window, global and random attention) sparse pattern for each block by the sorting of accumulative scores of row segment, column segment and diagonal segment in the block, leading to medium prediction complexity and memory overhead. As the generated sparse pattern remains regular to a certain extent, it could be efficiently accelerated via PEAs. However, as every block retains similar patterns, even for those unimportant blocks, it can only achieve a medium level of sparsity under the same accuracy level.

**Block-wise threshold-based pruning.** Accelerators that adopt block-wise threshold-based pruning also conduct value pruning within blocks via an empirical threshold. As the comparison with thresholds involves the entire row, this kind of pruning has the same advantages and disadvantages as the category of row-wise threshold-based pruning, and there is no found work falling in this category.

**Group-wise sorting-based pruning.** Accelerators using group-wise sorting-based pruning exploiting sparsity via value sorting within a *group*, which is much smaller than an entire *row* or a *block*. The typical work of this category is dynamic N:M pruning [6], which selects  $N$  largest ones from  $M$  consecutive scores of a group. As value sorting is performed in a small group and generates only  $N$  indices from  $M$ , it has a low prediction complexity and memory overhead. In

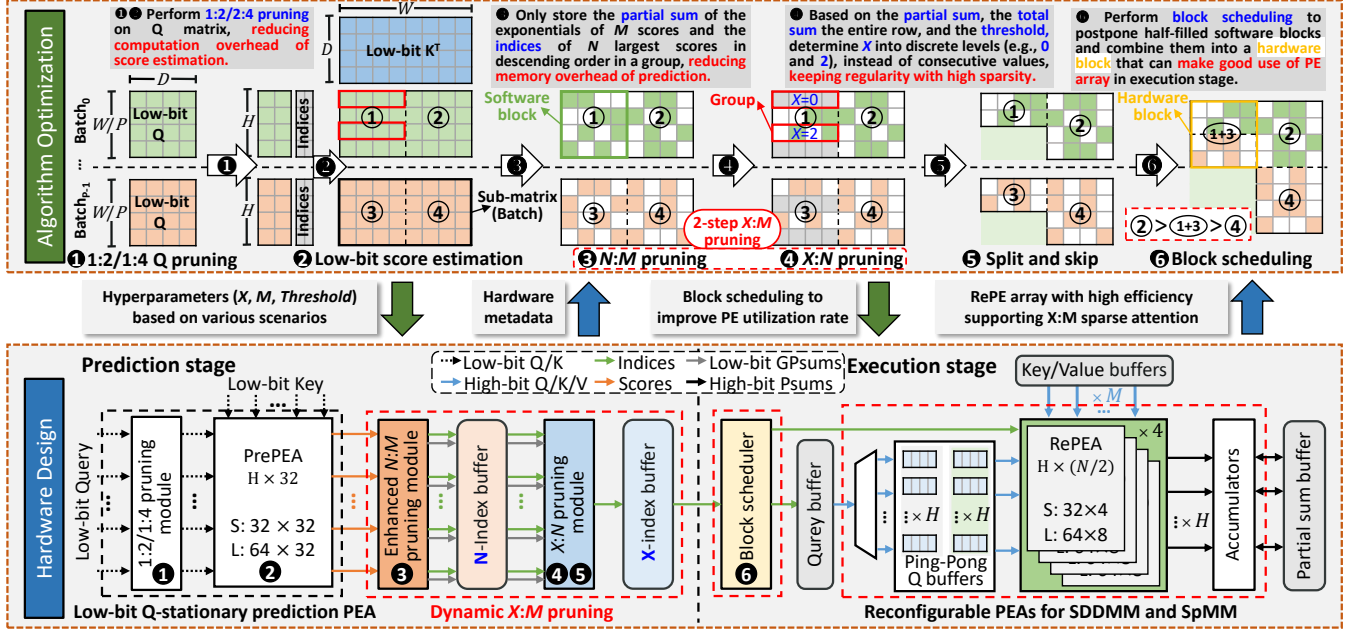


Figure 3. DynaX framework overview.

addition, as every group includes the same number ( $N$ ) of scores, the generated patterns remain a high regularity. However, as important and unimportant groups holds the same number of scores, it is difficult to achieve a high sparsity while retaining high accuracy.

**Group-wise threshold-based pruning.** The potential accelerators adopting group-wise threshold-based pruning can flexibly select a variable number ( $X$ ) of scores from a group (with the size of  $M$ ) based on a threshold, denoted by vanilla  $X:M$ . Like other threshold-based pruning techniques, this category needs normalization on the entire rows, leading to heavy memory overhead for prediction and irregularity of sparse patterns. Consequently, there is also no published work found of this category.

From the taxonomy above, we note that group-wise pruning keeps low complexity for prediction, which helps to scale sparse attention to long sequences. *In the scope of group-wise pruning,  $N:M$  sparse attention can maintain low prediction complexity, low memory overhead, and high regularity while vanilla  $X:M$  sparse attention is promising to achieve high sparsity.* If we can reinforce the strengths of both paradigms, it is promising to achieve a high throughput and energy-efficient sparse attention accelerator. To this end, we propose a new mechanism called *dynamic structured  $X:M$  sparse attention*, which not only adopts threshold-based pruning to achieve high sparsity but also keeps the regularity of  $N:M$  to a certain extent for efficient hardware implementation.

### 2.3 Challenges of Structured $X:M$ Sparse Attention

From discussion above, we find that dynamic structured  $X:M$  sparse attention mechanism has the potential to achieve a good balance among sparsity, prediction cost and hardware friendliness. However, it is challenging to achieve the potential for two reasons.

1) How to reduce the memory overhead of prediction when using threshold-based pruning? Although threshold-based pruning can achieve high sparsity, it involves large memory to store the approximate scores for reduce operation (softmax), which becomes unacceptable when scaling to long sequences. To address this challenge, we introduce a 2-step  $X:M$  pruning method that can significantly reduce the prediction memory overhead.

2) How to keep regularity for efficient hardware implementation with a variable  $X$ ? Although the variable  $X$  results in benefit of high sparsity, it is prone to unstructured sparse pattern, leading to inefficient hardware for SDDMM and SpMM. To address this challenge, we first constrains  $X$  to 3 discrete integers, instead of arbitrary integers in  $[0, M]$ . Then, we propose a block scheduling algorithm to reform *hardware blocks* that can make good use of PEA for SDDMM and SpMM.

## 3 DynaX Overview

Figure 3 presents an overview of the framework of our proposed DynaX which performs algorithm-hardware co-design for low prediction cost, and high sparsity while preserving structured pattern.

From the algorithm perspective, ❶ we first perform 1:2/1:4 fine-grained structured pruning on the low-bit  $Q$  matrix. Then, based on the low-bit  $K$  matrix and the compressed low-bit  $Q$  matrix, ❷ we perform matrix multiplication to generate the low-bit score matrix in a batch-by-batch fashion. Considering the constraint of memory, the target score matrix is horizontally divided into  $P$  sub-matrices (i.e., *batches*) with size of  $H \times W$  ( $H = W/P$ ) for score estimation, where  $H$  and  $W$  are height and width (i.e., sequence length) of the batch, respectively. Next, ❸ we conduct N:M fine-grained structured pruning on each batch in block-by-block fashion, where the batch is further vertically divided into  $G$  *software blocks* (SWBlock) with widths of  $M$ . After that, ❹ we do X:N pruning on each group for further pruning. Next, ❺ we perform split-and-skip on each software block and generate *shrank software blocks* (most groups are skipped) or *expanded software blocks* (most groups are split). Finally, ❻ we perform block scheduling such that these shrink software blocks can be postponed and combined into *hardware blocks* (HWBlock) that can match the PEA size for SDDMM and SpMM.

From the hardware perspective, we propose an architecture to efficiently support the optimized algorithm above, which works in two stages (prediction stage and execution stage). In the prediction stage, DynaX first uses a Query-stationary prediction PEA (PrePEA) to calculate the approximate scores, where 1:2/1:4 pruning modules are used to get the maximal value and its relative position out of 2/4 consecutive Query values. Then, the enhanced N:M module is used to select  $N$  largest scores out of  $M$ , perform exponential and accumulation operations, and finally store the  $N$  indices and the group partial sum (GPsum) into a N-index buffer. Next, X:N pruning module is adopted to further select  $X$  indices from  $N$  and store them in an X-index buffer. In the execution stage, a block scheduler is used to reorganize software blocks into hardware blocks to improve the PE utilization rate. Finally, a reconfigurable PE array (RePEA) is proposed to perform SDDMM and SpMM operations.

## 4 Algorithm Optimization

To exploit the dynamic fine-grained structured X:M sparsity on the score matrix, we propose a series of optimizations from the perspective algorithm, including 1:2/1:4 pruning of low-bit  $Q$  matrix, 2-step X:M pruning of score matrix and block scheduling algorithm to reorganize software blocks into hardware blocks.

### 4.1 Attention Score Estimation with Quantization and 1:2/1:4 Structured Pruning on $Q$ Matrix

To reduce the pre-computation cost of scores, the most common practices are to conduct low-bit quantization (e.g., 4-bit quantization) on the input matrix of  $Q$  and  $K$  such that the corresponding PEs can be greatly reduced. However, when the feature size becomes long, e.g., 128 for most LLMs, it still

leads to a large number of multiply-accumulate (MAC) units. Thus, besides the quantization operation, we also introduce 1:2/1:4 fine-grained structured pruning to the low-bit  $Q$  matrix, reducing the number of PEs needed. We identify that short-sequence tasks with a feature size of 64 are not sensitive to the calculation precision of attention scores but are highly sensitive to the sparsity of  $Q$ . Instead, long-sequence tasks with a feature size of 128 are sensitive to the calculation precision of attention scores but are not sensitive to the sparsity of  $Q$ . Therefore, for short-sequence tasks, we use a 1:2 sparse  $Q$  with 4-bit quantization, and for long-sequence tasks, we use a 1:4 sparse  $Q$  with 6-bit quantization. With  $Q$ -stationary data flow, the PrePEAs for both short-sequence tasks and long-sequence tasks maintain the same width of 32.

### 4.2 Two-step X:M Pruning

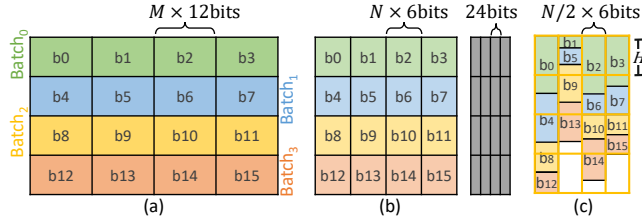
To address the challenge of prediction memory overhead when using threshold-based pruning, we propose a 2-step X:M structured pruning method, including N:M pruning and X:N pruning. In the step of N:M pruning, we perform top- $N$  sorting and select  $N$  largest scores from  $M$  scores, which can be quickly achieved since the group size ( $M$ ) is relatively small. After that, instead of storing the  $N$  largest scores like traditional N:M pruning, we only store the corresponding in-group relative indices in descending order of corresponding scores, which can significantly reduce memory footprint since the bit-width of relative indices (e.g., 6 bits) is much less than that of scores (e.g., 12 bits). In addition, we also store the partial exponential sum of  $M$  scores for each group for further pruning. After the scores of the entire row are generated, we start to perform X:N pruning to further improve the sparsity. To determine the value of  $X$ , we first define the *group importance* ( $\zeta$ ) for the  $g^{th}$  group in the  $i^{th}$  row as follows:

$$\zeta_{i,g} = \frac{\sum_{j=g \cdot M}^{(g+1) \cdot M - 1} e^{S_{i,j}}}{\sum_{j=0}^{W-1} e^{S_{i,j}}} \cdot \frac{W}{M} \quad (1)$$

where  $W$ ,  $M$  and  $S_{i,j}$  indicate the width of the score matrix, the group size and the estimated scores in the  $i^{th}$  row and  $j^{th}$  column, respectively. Based on the group importance and two empirical thresholds ( $T_0$  and  $T_1$ ), we further determine  $X$  value as follows:

$$X_{i,g} = \begin{cases} 0, & \zeta_{i,g} < T_0 \\ N/2, & T_0 \leq \zeta_{i,g} < T_1 \\ N, & \zeta_{i,g} \geq T_1 \end{cases} \quad (2)$$

where  $N$  is a hyper-parameter of previous N:M pruning. In the case of  $\zeta_{i,g} < T_0$ , this group is unimportant and all the scores in the group are pruned. In the case of  $\zeta_{i,g} \in [T_0, T_1)$ , the group is medium important and half of the scores in the group are pruned based on the result N:M pruning. In the case of  $\zeta_{i,g} \geq T_1$ , the group is important and all  $N$  scores are retained. As the residual groups ( $N/2$  and  $N$ ) after X:M



**Figure 4.** Illustration of X:M pruning. (a) The attention score matrix after estimation, (b) the N-index matrix and group partial sum matrix after N:M pruning, and (c) the X-index matrix after X:N pruning and skip-and-split.

pruning keep double relationships, the important group with size of  $N$  can be easily split into 2 medium important groups with size of  $N/2$ . Thus, all processed group finally has the same number of non-zeros ( $N/2$ ), maintaining regular sparse patterns to a certain extent.

As shown in Figure 4, assuming that the bit-widths for the estimated score, the in-group index and the group partial sum are 12 bits, 6 bits and 24 bits, the compression ratio of our proposed enhanced N:M pruning is  $\frac{12M}{6N+24}$ . When setting  $M$  and  $N$  to 64 and 16, the compression ratio reaches 6.4, significantly reducing the prediction memory overhead.

### 4.3 Block Scheduling on X-index Matrix

After X:M pruning, although split-and-skip can maintain the same width ( $N/2$ ) for different *software blocks*, as shown in Figure 4(c), they still exhibit different heights. As the RePEA for SDMM/SpMM has a fixed size, shrunk software blocks will not make good use of the PEA while the expanded software blocks will exceed the available resources.

To address this issue, we propose a block scheduling algorithm that can reorganize software blocks into hardware blocks to make good use of PEA under the constraints of memory size. As shown in algorithm 1, it takes parameters of hardware ( $H$  and  $R$ ) and score matrix ( $S$  and  $G$ ) as inputs and takes two issue buffers ( $IssueBuf_0$  and  $IssueBuf_1$ ) of hardware blocks as outputs. To free up the storage space of  $Q$ -matrix memory as soon as possible, hardware blocks consisting of multiple software blocks will be pushed into  $IssueBuf_0$  for priority execution while other blocks will be pushed into  $IssueBuf_1$ . The algorithm starts with the initialization of residual software blocks ( $ResBlocks$ ) and the number ( $ResBlockNum$ ) from previous batches. Then, for each batch in the score matrix, we conduct N:M pruning (③) and generate the N-index blocks ( $NBlocks$ ). Next, for each block ( $Nblock$ ) in  $NBlocks$ , we perform X:N pruning and skip-and-split, and get the pruned block ( $Xblock$ ). Based on the shape of  $Xblock$  and the residual blocks ( $ResBlocks$ ) with the same in-batch  $ID$ , we conduct scheduling in a block-by-block fashion (line 8-38). For each  $Xblock$ , we first fuse it into the corresponding residual blocks ( $ResBlocks[ID]$ )

#### Algorithm 1: Block scheduling algorithm.

---

**Input:**  $H$ : the height of RePEA;  $S$ : the estimated score matrix;  $G$ : the number of software blocks in a batch;  $R$ : the maximal number of batches that can be held in the  $Q$ -matrix buffer.

**Output:**  $IssueBuf_0$ ,  $IssueBuf_1$ : the scheduled hardware blocks.

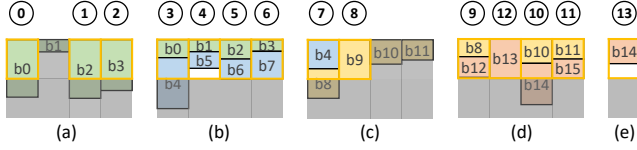
```

1  $ResBlockNum[G] \leftarrow \{0, 0, \dots, 0\}$ ;
2  $ResBlocks[G] \leftarrow \{0, 0, \dots, 0\}$ ;
3 for each batch of the score matrix  $S$  do
4    $Nblocks \leftarrow$  Perform N:M pruning on the batch ③;
5   for each  $Nblock$  in  $Nblocks$  do
6      $Xblock \leftarrow$  Perform X:N pruning on  $Nblock$  ④;
7      $Xblock \leftarrow$  Perform skip-and-split on  $Xblock$  ⑤;
8     /* scheduling for hardware blocks ⑥ */
9      $ID \leftarrow$  GetIntraBatchBlockID( $Xblock$ );
10     $ReadyBlocks \leftarrow ResBlocks[ID] + Xblock$ ;
11     $height \leftarrow$  GetHeight( $ReadyBlocks$ );
12    if  $height \geq H$  then
13      while  $height \geq H$  do
14         $HWBlock \leftarrow$  Truncate( $ReadyBlocks$ ,  $H$ );
15        if  $ResBlockNum[ID] + 1 \geq R$  then
16           $IssueBuf_0 \leftarrow IssueBuf_0 + HWBlock$ ;
17        end
18        else
19           $IssueBuf_1 \leftarrow IssueBuf_1 + HWBlock$ ;
20        end
21         $ResBlocks[ID] \leftarrow ReadyBlocks - HWBlock$ ;
22        if  $height > H$  then
23           $ResBlockNum[ID] \leftarrow 1$ ;
24           $ReadyBlocks \leftarrow ResBlocks[ID]$ ;
25           $height \leftarrow$  GetHeight( $ResBlocks[ID]$ );
26        else
27           $ResBlockNum[ID] \leftarrow 0$ ;
28        end
29      end
30    else if  $ResBlockNum[ID] + 1 \geq R$  then
31       $IssueBuf_0 \leftarrow IssueBuf_0 + ReadyBlocks$ ;
32       $ResBlocks[ID] \leftarrow 0$ ;
33       $ResBlockNum[ID] \leftarrow 0$ ;
34    end
35    else
36       $ResBlocks[ID] \leftarrow ReadyBlocks$ ;
37       $ResBlockNum[ID] \leftarrow ++1$ ;
38    end
39  end
40 end
41  $IssueBuf_1 \leftarrow IssueBuf_1 +$  All residual blocks;

```

---

and obtain the ready blocks ( $ReadyBlocks$ ). If the height of  $ReadyBlocks$  is greater than or equal to the height of RePEA ( $H$ ), we iteratively perform truncation on  $ReadyBlocks$  with  $H$  to get a hardware block ( $HWBlock$ ), push it to the issue buffers accordingly (line 13-19), and update  $ReadyBlocks$  and  $ReadyBlockNum$  (21-27) until the height of the residual block became less than  $H$ ; otherwise, if the residual blocks consisting more than or equal to  $R$  software blocks, we directly push the fused ready blocks to  $IssueBuf_0$ , resulting



**Figure 5.** Illustration of block scheduling. (a) Scheduling in the 1<sup>st</sup> batch, (b) scheduling in the 2<sup>nd</sup> batch, (c) scheduling in the 3<sup>rd</sup> batch, (d) scheduling in the 4<sup>th</sup> batch, and (e) scheduling for the residual blocks.

in higher execution priority (line 30-34); otherwise, we do not consider to issue the fused blocks in this round and update *ResBlocks* and *ResBlockNum*, accordingly (line 35-38). During the scheduling process above, once one of the issue buffers become non-empty, the RePEA can fetch hardware blocks from them for further execution, where *IssueBuf<sub>0</sub>* always keeps the higher priority.

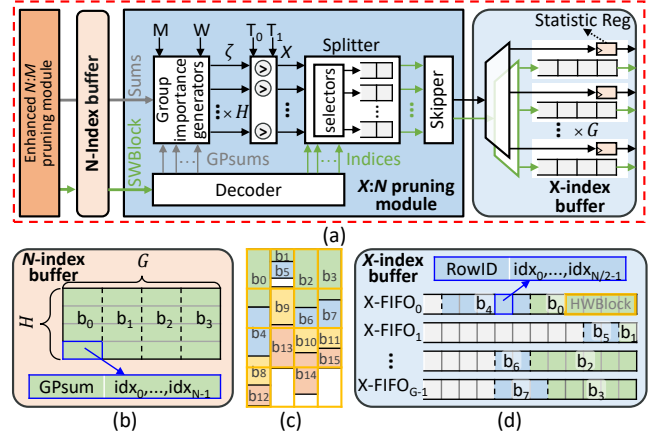
Figure 5 gives an example to illustrate the process of the scheduling algorithm for the compressed matrix in Figure 4(c), where the yellow boxes and the circled number indicate the formed hardware blocks and the corresponding issuing priority, respectively. It assumes that the maximum number of batches supported by the *Q*-matrix buffer is 2 ( $R = 2$ ). In the 1<sup>st</sup> batch, software blocks  $b_0, b_2, b_3$  reach the RePEA height and will be partially executed (matching the RePEA height). In the 2<sup>nd</sup> batch, the software block composed of  $b_1$  and  $b_5$ , although not reaching the height of RePEA, is forcibly issued due to storage limitations ( $R=2$ ). Please note that in the 4<sup>th</sup> batch,  $b_{13}$  will be the last one to be issued, because all other software blocks contain two blocks and are in the higher priority. From this example, we note that most hardware blocks are full except for the 4<sup>th</sup> and the last one, resulting in a high utilization rate of the RePEA in SDMM/SpMM stage.

## 5 Hardware Design

In this section, we mainly introduce 3 hardware modules of DynaX: the X:M pruning module, the block scheduling module, and the RePEA.

### 5.1 The Implementation of X:M Pruning

Figure 6 presents the hardware architecture to achieve X:M pruning, which consists of an enhanced N:M pruning module, an N-index buffer, an X:M pruning module, and an X-index buffer. Firstly, the enhanced N:M pruning module iteratively receives scores within a batch from the PrePEA, simultaneously calculates exponential partial sums (GPsum) of groups, selects  $N$  largest scores of groups, sorts these  $N$  scores in descending order, and finally stores the indices corresponding to the sorted  $N$  scores and the GPsums to the N-index buffer. As shown in Figure 6(b), the N-index buffer is a 1-bank SPM with width of  $H$  groups and depth of  $G$  that can hold 1 batch



**Figure 6.** The hardware implementation of X:M pruning. (a) Architecture overview of X:M pruning, (b) the data layout of N-index buffer that can hold 1 batch of N-indices, (c) the compressed X-index matrix example, and (d) the data layout of X-index buffer.

of N:M pruning results, where each group consists of a 24-bit GPsum and  $N$  5/6-bit intra-groups indices.

Based on the result from N:M pruning, the X:N pruning module further reduces  $N$  indices to  $X$  indices for each group. As shown in Figure 6(a), based on the decoded GPsums of a software block and the Row sums of a batch, the group importance generator generates  $H$  importance values ( $\zeta$ ) according to equation (1). Then, based on these importance values and two thresholds ( $T_0$  and  $T_1$ ), multiple comparators are used to generate the exact value of  $X$  for each group within the software block. Next, the splitter is used to select 0,  $N/2$  or  $N$  indices from  $N$  indices for each group and push the selected indices, with the unit of  $N/2$ , to  $H$  FIFOs with a depth of 2. If  $N$  indices are selected, they will be written to the FIFO in two cycles (*splitting* operation), separately. Finally, the skipper serially reads indices from  $H$  FIFOs of the splitter, and writes them to one of the X-index FIFOs (X-FIFOs) in the X-index buffer. Since FIFO read and write are performed serially, empty FIFO ( $X=0$ ) of the splitter can be trivially *skipped* and the X-FIFO will finally maintain a compact block of indices after splitting and skipping. As illustrated in Figure 6(c) and (d), the X-index buffer consists of  $G$  X-FIFOs with a depth of  $3H$ .  $G$  blocks of indices in a batch will be iteratively written to these X-FIFOs after X:N pruning. In addition, the X-index buffer also includes statistic registers recording how many software blocks (*BlockCnt*) and groups (*GroupCnt*) have been written into each X-FIFO. Each cell of an X-FIFO consists of a 16-bit row ID (RowID) and  $N/2$  5/6-bit indices. Since X-FIFO is filled with indices corresponding non-zeros, an hardware block (yellow box in Figure 6(d)) can be easily obtained by reading  $H$  consecutive data from it.

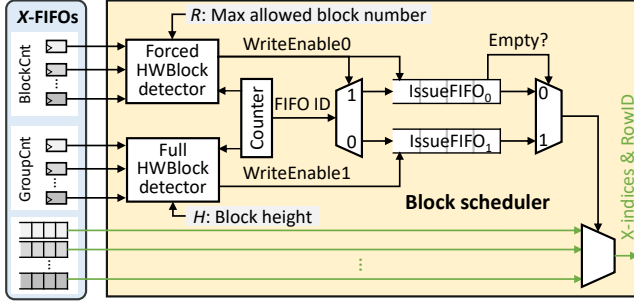


Figure 7. The hardware structure of block scheduler.

## 5.2 The Implementation of Block Scheduler

Figure 7 presents the hardware implementation of the block scheduler, which consists of a forced HWBlock detector, a full HWBlock detector, a counter, two issue FIFOs and some MUXs. Firstly, the counter iteratively generates the FIFO ID from 0 to  $G-1$ . For each FIFO ID, the forced HWBlock detector compares the corresponding block count to the maximal allowed block number to generate the write enable signal of Issue FIFO<sub>0</sub> ( $WriteEnable0$ ) with higher priority. Simultaneously, the full HWBlock detector compares the corresponding group count to the block height ( $H$ ) to generate the write enable signal of Issue FIFO<sub>1</sub> ( $WriteEnable1$ ). If  $WriteEnable0$  is valid, this FIFO ID will be pushed to issue FIFO<sub>0</sub>. Otherwise, this FIFO ID will be pushed to issue FIFO<sub>1</sub> if  $WriteEnable1$  is valid. If none of them are valid, no FIFO IDs will be pushed. If issue FIFO<sub>0</sub> becomes non-empty, FIFO ID will be read from it as priority and the HWBlocks will be read from the corresponding X-FIFO for further execution. If FIFO<sub>0</sub> is empty and FIFO<sub>1</sub> is non-empty, issue FIFO<sub>1</sub> will be read for further processing.

## 5.3 RePEA Architecture and Its Execution Pipeline

Figure 8(a) presents the hardware architecture of 4 RePEAs working in a pipelined fashion. Each RePEA consists of  $H \times (N/2)$  reconfigurable PEs (RePEs),  $H$  adder trees, and  $H$  rows of registers to latch the Keys or Values. Via these register rows, Key/Value groups flow into the RePEA in a systolic fashion. As to the Queries, since the RePEA for SDDMM works in data-flow of output (score) stationary and RePEs in the same row will consume the same query value, a Query from the ping-pong Q buffer is sent to the  $N/2$  RePEs within a row in a broadcast form, saving many cycles for feeding data. We don't need to take care of the scalability of broadcasting, because the RePEA width is relatively small (4 or 8) to match the width of HWBlock. To quickly obtain the partial sum, an adder tree is included in each row to add up the products from one row. Fig. 8(b), (c) and (d) further illustrated the datapaths for different phases within a RePE. In the first phase of SDDMM, based on the indices generated from the block scheduler, the RePE selects one Key from the

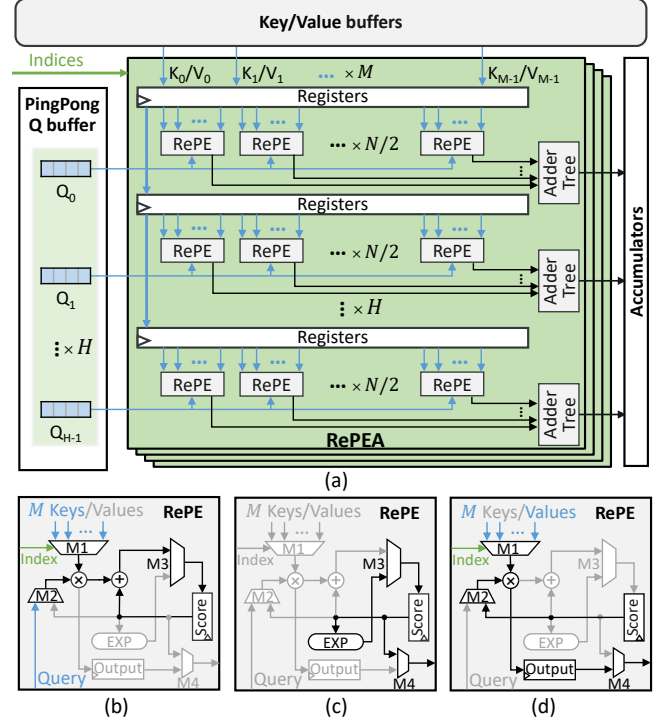


Figure 8. The implementation Reconfigurable PEA (RePEA). (a) The overall architecture of the RePEA, (b) the configured datapath in RePE for SpMM, (c) the configured datapath for exponential operation, and (d) the configured datapath for SDDMM.

register row (via M1), multiplies it to the Query, accumulates the products, and finally stores the partial sum into the score register (via M3). In the second phase, it performs exponential operation on the partial sum and updates the score register with the generated result. In the last phase of SpMM, the RePE selects one Value from the register row (via M1), multiplies it to the latched exponential score, and stores the product into the output register.

To reduce the bandwidth pressure of on-chip buffers, the RePEs are organized into 4 RePEAs, working in a pipelined fashion. As shown in Figure 9, the execution stage mainly has 5 sub-stages, including Query fetch (FetchQ), SDDMM, partial sum fetch (FetchPsum), SpMM and partial sum store (StorePsum). In these sub-stages, FetchQ involves data exchange between the Query buffer and the ping-pong Q buffer while FetchPsum/StorePsum involves data exchange between the accumulators and the partial sum buffer. Since the RePEA fetches data in a hybrid way of broadcast (horizontally) and systolic (vertically), the latency of executing one HWBlock in SDDMM/SpMM is determined by the systolic latency of Keys/Values, which is the sum ( $D + H$  in cycles) of feature size ( $D$ ) and RePEA height ( $H$ ). As to FetchQ, FetchSum, and StoreSum, they all have a latency of  $H$  cycles since the movement of Queries/Partial sums corresponding to one PE

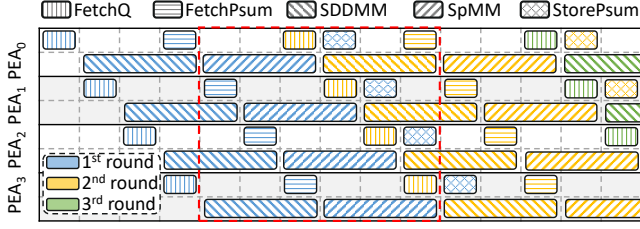


Figure 9. The execution pipeline of the 4 RePEAs.

row consumes one cycle and the movements for different rows are sequentially performed. Thus, the latency of SDDMM/SpMM is about  $3 \times$  of FetchQ/StorePsum assuming that  $D$  is twice of  $H$ . In the kernel of pipeline, as the red dashed box depicted in Figure 9, the 4 RePEAs in SpMM sub-stage will consume  $4 \times H \times N/2$  scores (each RePE hold a score) in  $6H$  cycles; in other words, the 4 RePEAs will consume  $N/3$  scores (after pruning) per cycle at most. As to the prediction PEA, it works in the fashion of Query stationary and it can generate  $H$  dense scores per cycle at most. Therefore, the maximal sparsity supported by DynaX is  $1 - N/(3H)$ . In the actual implementation,  $H$  is  $8 \times$  of  $N$ . Consequently, DynaX can support the maximal sparsity of  $1 - 1/24 \approx 95.8\%$ , which can cover the majority of scenarios.

## 6 Evaluation

### 6.1 Experiments Setup

**Hardware Implementation.** As most existing works only have evaluation results on short sequences, we implemented two hardware versions, DynaX-S and DynaX-L, for fair comparison. DynaX-S for short-sequence tasks is equipped with a  $32 \times 32$  PrePEA (4-bit multipliers) with 1:2 low-bit Q pruning, and four  $32 \times 4$  RePEAs. DynaX-L for long-sequence tasks is equipped with one  $64 \times 32$  PrePEA (6-bit multipliers) with 1:4 low-bit Q pruning and four  $64 \times 8$  RePEAs. We performed RTL implementation of DynaX-S and DynaX-L hardware, synthesized using 28nm CMOS technology in Synopsys Design Compiler with a frequency of 500MHz. We also implemented a cycle-level simulator, with input data sourced from the attention matrices generated by the model during inference, to evaluate the throughput and energy efficiency of DynaX.

**Benchmarks.** We evaluated DynaX using the open-source models BERT-base [11], Llama3-8B [14], Bloom-7B1 and Bloom-1B7 [22] across 13 tasks. The BERT model was used to evaluate DynaX-S, with 8 tasks from the general language understanding evaluation (GLUE) [49] datasets (linguistic acceptability CoLA [52], sentiment analysis SST-2 [45], sentence similarity and paraphrasing MRPC [12], QQP and STS-B [4], natural language inference MNLI [53], question answering QNLI [42], and textual entailment RTE [8]), the Stanford Question Answering Dataset v1.1 (SQuAD) [42], and the Cloze Test by Teachers (CLOTH) [55]. The Llama3

Table 1. Area and power breakdown of DynaX.

| Module             | DynaX-S / L<br>Area [mm <sup>2</sup> ] | DynaX-S / L<br>Power [mW] |
|--------------------|----------------------------------------|---------------------------|
| Prediction Array   | 0.10 / 0.43                            | 23.42 / 104.79            |
| X:M Pruning Module | 0.02 / 0.07                            | 5.23 / 20.93              |
| RePEAs             | 0.62 / 2.79                            | 98.23 / 393.62            |
| Memory Module      | 0.26 / 2.61                            | 38.77 / 348.93            |
| Other Module       | 0.07 / 0.15                            | 18.82 / 37.65             |
| <b>Total</b>       | <b>1.08 / 6.05</b>                     | <b>184.47 / 905.92</b>    |

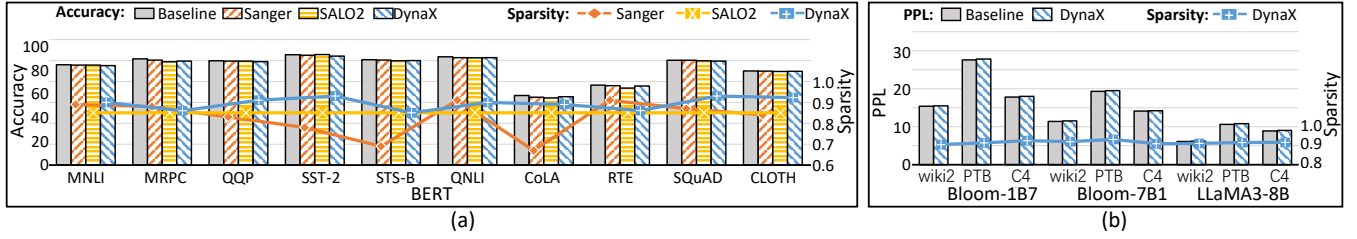
and Bloom model were used to evaluate DynaX-L, with tasks including wikitext-2 (wiki2) [33], Penn Treebank (PTB) [31] and a portion of the C4 dataset [41]. For tasks MRPC, QQP, and SQuAD, we report the F1 score, and for tasks wiki2, PTB and C4, we report perplexity (PPL). Note that PPL is negatively correlated with model performance, while F1 is positively correlated.

**Software Implementation.** We implemented the attention sparsity method of DynaX using Pytorch [35] and the Transformers library of HuggingFace [54]. In our experiments, the N:M for DynaX-S was set to 8:32, and for DynaX-L it was set to 16:64. By the exploration of  $T_0 \in [0.02, 0.6]$  and  $T_1 \in [0.2, 3]$ , we achieve comparable accuracy with dense attention. Please note that to ensure fairness, after determining the hyperparameters, we fine-tune and evaluate the models using the same learning rate, number of epochs, and batch size as the baseline. The sparsity in our experiments is determined by averaging the statistics of all attention heads for the test set of each task.

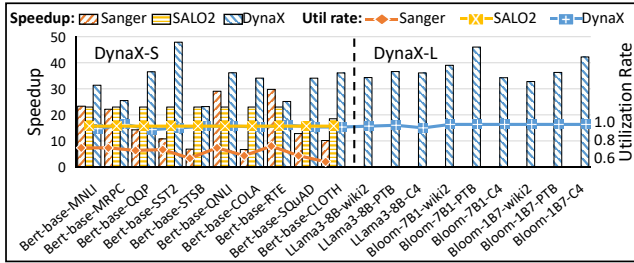
**Hardware Baselines.** We compared DynaX with Sanger [28], SALO2 [61], and the NVIDIA V100 SXM2 GPU, where Sanger and SALO2 are open-source accelerators. To ensure fairness, we scaled down the RePEA of Sanger to match the size of DynaX-S ( $32 \times 4 \times 4$ ). Compared to the original array size, this will somewhat increase its energy efficiency. When comparing with the GPU, we scaled up DynaX, Sanger and SALO2 to match the tensor peak throughput (125 TOPS) of V100 and re-simulated the throughput and energy efficiency.

### 6.2 Area and Power Consumption of DynaX

As shown in Table 1, the power consumption of DynaX-S and DynaX-L is 184.47 mW and 905.92 mW, with the main overhead coming from the RePEAs and SRAM. DynaX-S has 172 KB of on-chip storage, supporting an input sequence length of 1024 and a single-head dimension of 64, including 128 KB K/V buffer, 24 KB Q/partial sum buffer and 20 KB indices buffer. The larger DynaX-L has 2447 KB of on-chip storage, supporting an input sequence length of 8192 and a single-head dimension of 128, including 2048 KB K/V buffer, 96 KB Q/partial sum buffer and 303 KB index buffer.



**Figure 10.** Model performance comparison. (a) Performance on short-sequence tasks of BERT, and (b) performance on long-sequence tasks of Bloom-1B7, Bloom-7B1 and LLaMA3-8B.



**Figure 11.** The normalized speedup of Sanger, SALO2, and DynaX comparison with GPU.

### 6.3 Model Performance Evaluation

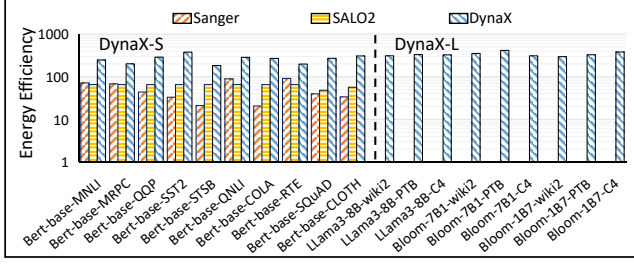
Figure 10 presents the model performance in terms of accuracy (in F1 or PPL) and sparsity for different tasks, where baseline indicates dense attention. For fair comparisons, DynaX tries to achieve comparable accuracy with Sanger and SALO2 by adjusting the thresholds ( $T_0$  and  $T_1$ ) and then compares the sparsity with them. After threshold exploration of X:M pruning, as the bars shown in Figure 10(a), DynaX can achieve an average accuracy of 82.23 on BERT model over short-sequence tasks, which is about 0.99% loss from the baseline and comparable with those of Sanger (82.57) and SALO2 (82.08). As to the sparsity, as the lines shown in Figure 10(a), DynaX can achieve an average sparsity of 89.54%, which is about 1.08 $\times$  and 1.05 $\times$  of Sanger (82.70%) and SALO2 (85.00%), respectively. When transferring the sparsity into the proportion of non-zeros, DynaX only has 60.46% and 69.73% non-zero scores of Sanger and SALO2, respectively. When moving to models for long-sequence tasks, Sanger and SALO2 can hardly work (no reported data) for their high prediction cost. As to DynaX, as shown in Figure 10(b), it can achieve an average sparsity of 91.77% under an average PPL of 14.72, which is only 0.98% accuracy loss compared to the baseline. In summary, owing to the flexible selection of X in X:M pruning, DynaX can achieve average sparsity of 89.54% and 91.77% for short-sequence tasks and long-sequence tasks, respectively, with less than 1% accuracy loss compared to dense attention.

### 6.4 Speedup Evaluation

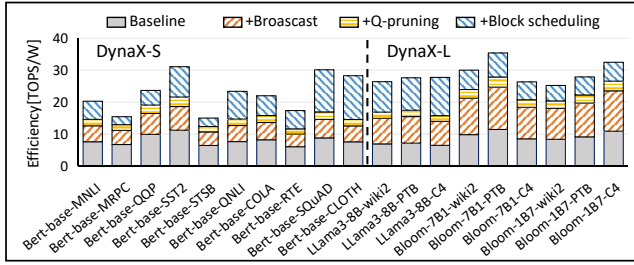
Figure 11 shows the normalized speedup (to V100 GPU) of DynaX, Sanger, and SALO2. We set a sufficiently large batch size for each task to ensure that the GPU operates at near full load to achieve maximum throughput. For the evaluation of Sanger and SALO2, we also scaled them to have comparable throughput with the V100 and recalculated the normalization according to their published papers. On average, DynaX-S achieves 33.00 $\times$  speedup on BERT while DynaX-L achieves 37.51 $\times$  speedup on LLMs, as compared to the GPU. When focusing on BERT, DynaX-S accelerates on average by 1.99 $\times$  compared to Sanger. Besides the higher sparsity, the main reason is that DynaX can generate regular hardware blocks matching the RePEA size via careful selection of X (0,  $N/2$  and  $N$ ) and block scheduling. Consequently, as the lines shown in Figure 11, our utilization rate across all tasks exceeded 93%, with an average utilization rate reaching 97%, which is significantly higher than the 67% average utilization rate of Sanger. As to SALO2 on BERT, with comparable utilization rates, DynaX-S accelerates on average by 1.55 $\times$ , which is due in large part to our higher sparsity. However, SALO2 is difficult to achieve a good balance between sparsity and utilization rate. As discussed in [61], when sparsity is increased from 85% to 87%, the utilization rate of SALO2 drops by more than 17%, which would significantly reduce the hardware throughput. Overall, thanks to the X:M pruning and block scheduler, DynaX can maintain regular sparse patterns and higher utilization rates, resulting in high throughput.

### 6.5 Energy Efficiency Evaluation

Figure 12 gives the comparison of normalized (to V100) energy efficiency of Sanger, SALO2 and DynaX. For the power of Sanger and SALO2, we synthesized their open-source code under the same frequency (500MHz) and technology (28nm). As can be seen in Figure 12, DynaX-S achieved 5.16 $\times$ , 4.20 $\times$  and 263.32 $\times$  the energy efficiency improvement over Sanger, SALO2 and V100 on BERT. In addition to the advantages on the software side (high sparsity), the energy efficiency improvement of DynaX over Sanger comes from the high utilization rate of PEs and the efficient RePEA, as the bubble controller in Sanger accounts for more than half of the entire



**Figure 12.** Energy efficiency improvement of Sanger, SALO2, and DynaX comparison with GPU.



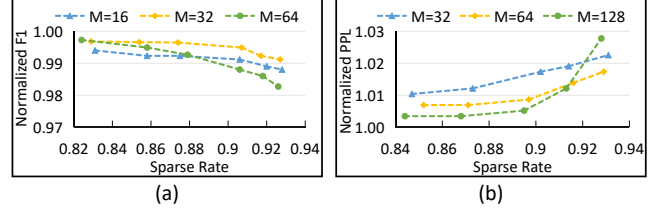
**Figure 13.** Energy efficiency improvement breakdown of DynaX.

computing array overhead; the energy efficiency improvement over SALO2 comes from the low-overhead prediction array and regular SRAM reading, as the prediction mechanism of SALO2 is overly complex and data cannot systolic when computing random attention. DynaX-L achieved better results on the LLaMA3 and Bloom models because long sequences can maintain model performance at higher sparsity levels. Compared to the V100 GPU, DynaX-L achieved energy efficiency improvements of  $339.13 \times$ .

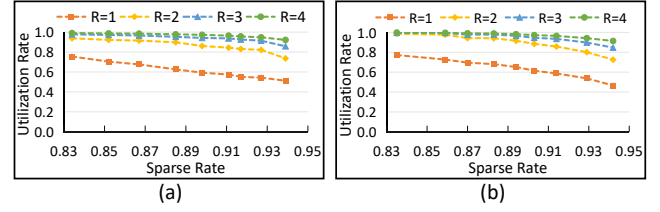
Figure 13 illustrates the energy efficiency breakdown of DynaX, where the improvement in energy efficiency comes from three aspects. First, the horizontal broadcasting of the RePEA eliminates the majority of registers used for position control, reducing the execution array overhead by 30.14% (DynaX-S) and 43.03% (DynaX-L). Particularly, the broadcast in DynaX-L has increased from 4 RePEs to 8, its larger array size requires more position control information, making the advantage of using broadcast in DynaX-L more pronounced. Second, the sparse prediction of DynaX achieves comparable or even better model performance with half the hardware of dense prediction via Q pruning. Third, the block scheduling algorithm, in conjunction with the X:M sparsity method, achieves an average utilization rate of 97.0% in DynaX. DynaX achieved an average energy efficiency of 25.55 TOPS/W for attention computation in all tasks.

## 6.6 Group Size Exploration

Since X:M is built on N:M, and  $N$  and  $M$  maintain a fixed ratio (i.e., 1:4), a larger group size of  $M$  would result in a larger  $N$ ,



**Figure 14.** Group size ( $M$ ) exploration over different sparsity. (a) BERT-base on SQuAD, and (b) LLaMA3-8B on wiki2.

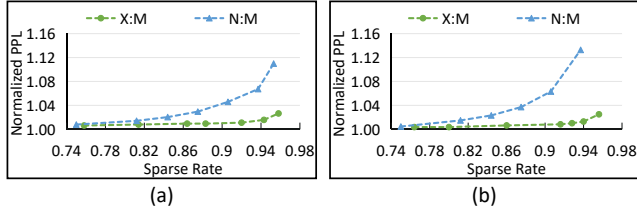


**Figure 15.** Query buffer size ( $R$ ) exploration over different sparsity. (a) BERT-base on SQuAD, and (b) LLaMA3-8B on wiki2.

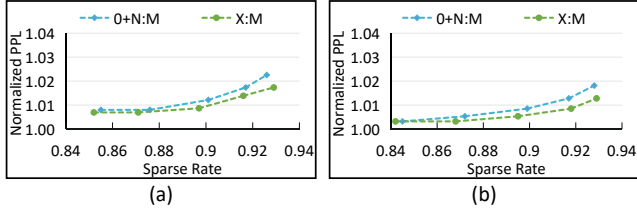
which not only leads to a larger memory for prediction but also incurs a wider RePEA for execution. Therefore, we need to reduce the group size as much as possible while achieving high sparsity. In Figure 14, we explored the different values of  $M$  for DynaX-S and DynaX-L using the BERT-base and LLaMA3-8B models, respectively. For the BERT-base model, with about 1% accuracy loss,  $M$  with values of 16, 32, and 64 achieve a sparsity of 90.6%, 92.7%, and 89.2%, respectively. Thus, we finally set  $M$  to 32 for DynaX-S. For the LLaMA3-8B model, with about 1% accuracy loss,  $M$  with values of 32, 64, and 128 achieve a sparsity of 84.4%, 90.3%, and 90.8%, respectively. Considering that there is only a slight improvement from 64 to 128, we finally set  $M$  to 64 for DynaX-L.

## 6.7 Query / Partial Sum Buffer Size Exploration

As discussed in algorithm 1, the buffer size of Query and Partial determines how many residual blocks ( $R$ ) could be retained to form a full hardware block. Since each residual block will occupy  $H \times D$  elements in the Query / partial sum buffer leading to heavy memory overhead, we need to reduce  $R$  as much as possible under the constraints of utilization rate. Figure 15 presents the impact of these storage configurations on hardware utilization using BERT and LLaMA3 model. It can be observed that for the BERT and LLaMA3 models, an  $R$  value of 4 is sufficient to maintain a utilization rate of over 96% in scenarios where sparsity is less than 90%, and over 92% when sparsity is less than 94%. Therefore, in our final implementation, we set  $R$  to 4 for DynaX.



**Figure 16.** Model performance of LLaMA3-8B on (a) wiki2 and (b) PTB with X:M and N:M pruning.



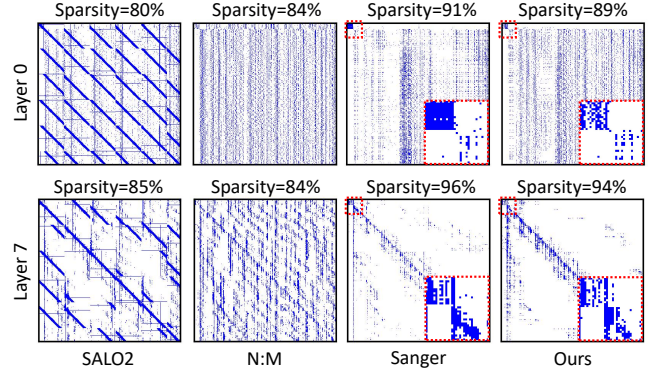
**Figure 17.** Model performance of LLaMA3-8B on (a) wiki2 and (b) PTB using 3-level X:M and 2-level X:M (0:M+N:M).

### 6.8 Effectiveness of structured X:M versus N:M

Since our proposed structured X:M pruning is built on traditional N:M pruning, we compare the performance of N:M and X:M on PPL and sparsity to show our effectiveness. In Figure 16, we explored the performance of N:M and X:M at different sparsity levels on the wiki2 and PTB of LLaMA3. Please note that to ensure fairness, we directly sparsify the original attention scores in this part to exclude the model performance loss caused by the Q/K quantization and Q pruning. The group size ( $M$ ) for both methods is 64. For N:M pruning, the value of  $N$  is selected from 16 down to 3 for increased sparsity. For our X:M pruning, we adjust the thresholds for different sparsity letting  $X$  fall into 0/8/16 for different groups. It can be observed that as sparsity increases, N:M pruning struggles to maintain model performance, while X:M still achieves high model performance (low PPL). With 1% performance loss, X:M can achieve an average sparsity of 92.4% while N:M only has an average sparsity of 81.2%.

### 6.9 Effectiveness of 3-level X:M versus 2-level X:M

The level of X:M pruning is closely related to model accuracy and sparsity. In Figure 17, we compared the model performance of 2-level X:M (i.e., 0:M and N:M) and 3-level X:M (i.e., 0:M, N/2:M and N:M) over different sparsity levels. With about 1% accuracy loss, the average sparsity for 2-level X:M is 89.6%, while for 3-level X:M, it is 91.2%. Although there is only 1.78% growth in sparsity, it will result in 15.4% reduction in operations of SDDM and SpMM. As to hardware implementation, 2-level X:M can almost share the same architecture of 3-level X:M by splitting all remaining groups ( $N$  elements) into  $N/2$ -element groups. Consequently, the 2-level X:M almost have the same hardware overhead as



**Figure 18.** Visualization of pruning results for different pruning methods on different layers of the BERT-base Model.

3-level X:M. Thus, we finally adopt 3-level X:M to perform software-hardware co-design.

### 6.10 Sparsity Distribution

Figure 18 illustrates the sparsity patterns of several pruning methods on layer 0 (it is sensitive to model accuracy and prefers conservative pruning), and layer 7 (it is robust to model accuracy and prefers aggressive pruning) of the BERT-base model. To ensure fairness, we adjusted hyperparameters for N:M, Sanger and X:M to achieve comparable model accuracy (0.4% 0.6% accuracy loss) while utilizing the reported parameters for SALO2. Without considering hardware resource utilization, Sanger utilizing row-wise threshold-based pruning can achieve the best results with global information, and thus, it is regarded as the ideal sparsity pattern. From Figure 18, we note that our method generates the closest pattern and sparsity as Sanger since we also use global information for pruning. Moreover, we also find that our method can adaptively conduct conservative pruning (about 89%) on sensitive layers while conducting aggressive pruning (about 94%) on other layers since the attention distribution of the sensitive layer is more even across all groups and our method can hardly find unimportant groups for aggressive pruning. Compared to Sanger, as the dashed boxes show, ours can maintain regularity patterns to a certain degree, i.e., each group has 0, 4 or 8 scores while Sanger may have an arbitrary number of scores in a group. Thus, our method is more friendly to hardware implementation, resulting in higher resource utilization.

### 6.11 Overall Assessment of DynaX

In addition to Sanger and SALO2, FACT [37] and Ayaka [36] are also state-of-the-art Transformer accelerators. Instead of focusing on attention computation only, FACT and Ayaka accelerate all three modules of the Transformer ( $QKV$  generation, attention, and FFN). For fair comparisons, we only focus on the attention computation part of FACT and Ayaka. Table 2 presents the comparison of area, power, frequency, energy

efficiency and area efficiency of these accelerators, where all of them are evaluated using 28 nm process. Moreover, throughput is not compared since there is no throughput of attention computation reported in [37] and [36]. When handling short-sequence tasks, compared to Sanger, SALO2 and FACT, the energy efficiency improvement of DynaX-S is  $5.16 \times$ ,  $4.20 \times$  and  $1.49 \times$ . As FACT computes sparse attention in a non-systolic manner, it needs a high-bandwidth on-chip network for data exchange, leading to more power consumption. When handling long-sequence tasks, DynaX-L achieves  $1.32 \times$  energy efficiency compared to Ayaka. Although Ayaka consumes less power with lower frequency and power gating, it has a low PE utilization rate (59.67%), resulting in lower throughput. Moreover, Ayaka also suffers from large area consumption, about  $1.77 \times$  of DynaX-L, even with half of our PEA size. Therefore, DynaX outperforms state-of-the-art accelerators in terms of energy efficiency.

## 7 Related Works

**Sparse Transformer Accelerators.** Numerous studies [25, 26, 32, 44, 47] have found that the attention mechanism involves a significant amount of unnecessary computations for less important scores, and explorations [6, 9, 15–18, 23, 24, 28, 36, 37, 40, 50, 57, 58, 61, 62] have been made in accelerating sparse attention. SpAtten [50] proposed a cascade of token pruning and head pruning to remove entire rows or columns of the attention matrix, or even entire attention heads, combined with row-level top-k pruning, which can reduce the computational cost of both attention and subsequent Linear layer, but such coarse-grained pruning cannot be applied to tasks with irregular attention distribution at high sparsity. DOTA [40] uses low-rank linear transformations to predict sparse attention masks and explores row-level parallelism and out-of-order execution of the attention matrix to increase on-chip data reuse, but compared to the systolic array approach, it still has a higher on-chip bandwidth. FACT [37] and Ayaka [36] focus on accelerating the entire Transformer module using row-wise sorting-based pruning for sparse attention. Targeting short-sequence tasks, attention computation is not the bottleneck for FACT and Ayaka. However, LLMs that support long sequences [14] have been widely applied, and the computation of attention will become the biggest bottleneck. Especially in some models with fewer weight parameters, such as LLaMA3.2-1B and Qwen2-1.5B [56], the dimensions of the weight matrix are far less than the sequence length. ViTCoD [58] focuses on accelerating Vision Transformers, using static masking to prune the attention matrix, which is not suitable for NLP tasks.

**Other Transformer Accelerators.** MECLA [38] noted the storage overhead issue of the weight matrix, scaling a larger weight matrix into two smaller source sub-matrices and derived sub-matrices through matrix partitioning, simultaneously reducing storage and computational overhead.

**Table 2. Comparison with other accelerators**

|                     | Area<br>[mm <sup>2</sup> ] | Power<br>[mW]        | Frequency<br>[MHz] | Energy Efficiency<br>[TOPS/W] |
|---------------------|----------------------------|----------------------|--------------------|-------------------------------|
| Sanger <sup>1</sup> | 3.24                       | 674.65               | 500                | 4.4                           |
| SALO2 <sup>1</sup>  | 2.64                       | 744.91               | 500                | 5.4                           |
| FACT <sup>2</sup>   | 6.03                       | 337.07               | 500                | 15.2                          |
| Ayaka <sup>2</sup>  | 10.76                      | 395.60               | 430                | 21.9                          |
| <b>DynaX-S/L</b>    | <b>1.08/6.05</b>           | <b>184.47/905.92</b> | <b>500</b>         | <b>22.7/28.8</b>              |

<sup>1</sup> Evaluation using open-source code at 500MHz with 28nm.

<sup>2</sup> We only focus on the part of attention computation.

However, MECLA requires the model to be retrained to reconstruct the weight matrix, which may be a huge overhead for many models, especially LLMs. FIGNA [21] noted that despite the memory footprint reduction achieved by weight quantization, the actual computing performance is not really improved due to dequantization from integer to float, and proposed a computational unit specifically designed for FP-INT MAC operations to improve computation performance of weights. DynaX is only concerned with the computation of attention, which is somewhat orthogonal to these two works to a certain extent.

## 8 Conclusions

This paper proposes an algorithm-hardware co-design framework, DynaX, for sparse attention acceleration. By introducing the mechanism of dynamic X:M fine-grained structured pruning, DynaX can achieve a good balance between accuracy, sparsity, and hardware friendliness. Experiments show that DynaX can achieve an average sparsity of 89.54% and 91.77% for short-sequence tasks and long-sequence tasks, respectively, with less than 1% accuracy loss. As compared to two state-of-the-art accelerators, DynaX-S and DynaX-L can achieve  $1.32 \times$  and  $1.49 \times$  energy efficiency, respectively.

## 9 Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 62274019, and the Fundamental Research Funds for the Central Universities (No. 2023CDJKYJH017). The corresponding author of this paper is Dajiang Liu (liudj@cqu.edu.cn).

## References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*, 2023.
- [3] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [4] Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. Semeval-2017 task 1: Semantic textual similarity-multilingual

- and cross-lingual focused evaluation. *arXiv preprint arXiv:1708.00055*, 2017.
- [5] Chaoqi Chen, Yushuang Wu, Qiyuan Dai, Hong-Yu Zhou, Mutian Xu, Sibe Yang, Xiaoguang Han, and Yizhou Yu. A survey on graph neural networks and graph transformers in computer vision: A task-oriented perspective. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
  - [6] Zhaodong Chen, Zheng Qu, Yuying Quan, Liu Liu, Yufei Ding, and Yuan Xie. Dynamic n: M fine-grained structured sparse attention mechanism. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 369–379, 2023.
  - [7] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
  - [8] Ido Dagan, Oren Glickman, and Bernardo Magnini. The pascal recognising textual entailment challenge. In *Machine learning challenges workshop*, pages 177–190. Springer, 2005.
  - [9] Jyotikrishna Dass, Shang Wu, Huihong Shi, Chaojian Li, Zhifan Ye, Zhongfeng Wang, and Yingyan Lin. Vitality: Unifying low-rank and sparse approximation for vision transformer acceleration with a linear taylor attention. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 415–428. IEEE, 2023.
  - [10] Mostafa Dehghani, Basil Mustafa, Josip Djolonga, Jonathan Heek, Matthias Minderer, Mathilde Caron, Andreas Steiner, Joan Puigcerver, Robert Geirhos, Ibrahim M Alabdulmohsin, et al. Patch n’pack: Navit, a vision transformer for any aspect ratio and resolution. *Advances in Neural Information Processing Systems*, 36, 2024.
  - [11] Jacob Devlin. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
  - [12] Bill Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In *Third international workshop on paraphrasing (IWP2005)*, 2005.
  - [13] Xiaoyi Dong, Jianmin Bao, Dongdong Chen, Weiming Zhang, Nenghai Yu, Lu Yuan, Dong Chen, and Baining Guo. Cswin transformer: A general vision transformer backbone with cross-shaped windows. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12124–12134, 2022.
  - [14] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
  - [15] Hongxiang Fan, Thomas Chau, Stylianos I Venieris, Royson Lee, Alexandros Kouris, Wayne Luk, Nicholas D Lane, and Mohamed S Abdelfattah. Adaptable butterfly accelerator for attention-based nns via hardware and algorithm co-design. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 599–615. IEEE, 2022.
  - [16] Chao Fang, Aojun Zhou, and Zhongfeng Wang. An algorithm–hardware co-optimized framework for accelerating n: M sparse transformers. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 30(11):1573–1586, 2022.
  - [17] Ruiqi Guo, Xiaofeng Chen, Lei Wang, Yang Wang, Hao Sun, Jingchuan Wei, Huiming Han, Leibo Liu, Shaojun Wei, Yang Hu, et al. Cim-former: A systolic cim-array-based transformer accelerator with token-pruning-aware attention reformulating and principal possibility gathering. *IEEE Journal of Solid-State Circuits*, 2024.
  - [18] Tae Jun Ham, Yejin Lee, Seong Hoon Seo, Soosung Kim, Hyunji Choi, Sung Jun Jung, and Jae W Lee. Elsa: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 692–705. IEEE, 2021.
  - [19] Ali Hassani, Steven Walton, Jiachen Li, Shen Li, and Humphrey Shi. Neighborhood attention transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6185–6194, 2023.
  - [20] Ronghang Hu and Amanpreet Singh. Unit: Multimodal multitask learning with a unified transformer. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1439–1449, 2021.
  - [21] Jaeyong Jang, Yulhwa Kim, Juheun Lee, and Jae-Joon Kim. Figna: Integer unit-based accelerator design for fp-int gemm preserving numerical accuracy. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 760–773. IEEE, 2024.
  - [22] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. Bloom: A 176b-parameter open-access multilingual language model. 2023.
  - [23] Bingbing Li, Santosh Pandey, Haowen Fang, Yanjun Lyv, Ji Li, Jieyang Chen, Mimi Xie, Lipeng Wan, Hang Liu, and Caiwen Ding. Ftrans: energy-efficient acceleration of transformers using fpga. In *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design*, pages 175–180, 2020.
  - [24] Zheng Li, Soroush Ghodrati, Amir Yazdanbakhsh, Hadi Esmaeilzadeh, and Mingu Kang. Accelerating attention through gradient-based learned runtime pruning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pages 902–915, 2022.
  - [25] Kevin Lin, Linjie Li, Chung-Ching Lin, Faisal Ahmed, Zhe Gan, Zicheng Liu, Yumao Lu, and Lijuan Wang. Swinbert: End-to-end transformers with sparse attention for video captioning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17949–17958, 2022.
  - [26] Liu Liu, Zheng Qu, Zhaodong Chen, Fengbin Tu, Yufei Ding, and Yuan Xie. Dynamic sparse attention for scalable transformer acceleration. *IEEE Transactions on Computers*, 71(12):3165–3178, 2022.
  - [27] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021.
  - [28] Liqiang Lu, Yicheng Jin, Hangrui Bi, Zizhang Luo, Peng Li, Tao Wang, and Yun Liang. Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 977–991, 2021.
  - [29] Arjun Majumdar, Anurag Ajay, Xiaohan Zhang, Pranav Putta, Sriram Yenamandra, Mikael Henaff, Sneha Silwal, Paul Mcvay, Oleksandr Maksymets, Sergio Arnaud, et al. Openeqa: Embodied question answering in the era of foundation models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16488–16498, 2024.
  - [30] Ben Mann, N Ryder, M Subbiah, J Kaplan, P Dhariwal, A Neelakantan, P Shyam, G Sastry, A Askell, S Agarwal, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 1, 2020.
  - [31] Mitch Marcus, Grace Kim, Mary Ann Marcinkiewicz, Robert MacIntyre, Ann Bies, Mark Ferguson, Karen Katz, and Britta Schasberger. The penn treebank: Annotating predicate argument structure. In *Human Language Technology: Proceedings of a Workshop held at Plainsboro, New Jersey, March 8–11, 1994*, 1994.
  - [32] Yiqun Mei, Yuchen Fan, and Yuqian Zhou. Image super-resolution with non-local sparse attention. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 3517–3526, 2021.
  - [33] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.
  - [34] Niki Parmar, Ashish Vaswani, Jakob Uszkoreit, Lukasz Kaiser, Noam Shazeer, Alexander Ku, and Dustin Tran. Image transformer. In *International conference on machine learning*, pages 4055–4064. PMLR, 2018.
  - [35] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017.

- [36] Yubin Qin, Yang Wang, Dazheng Deng, Xiaolong Yang, Zhiren Zhao, Yang Zhou, Yuanqi Fan, Jingchuan Wei, Tianbao Chen, Leibo Liu, et al. Ayaka: A versatile transformer accelerator with low-rank estimation and heterogeneous dataflow. *IEEE Journal of Solid-State Circuits*, 2024.
- [37] Yubin Qin, Yang Wang, Dazheng Deng, Zhiren Zhao, Xiaolong Yang, Leibo Liu, Shaojun Wei, Yang Hu, and Shouyi Yin. Fact: Ffn-attention co-optimized transformer architecture with eager correlation prediction. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–14, 2023.
- [38] Yubin Qin, Yang Wang, Zhiren Zhao, Xiaolong Yang, Yang Zhou, Shaojun Wei, Yang Hu, and Shouyi Yin. Mecla: Memory-compute-efficient llm accelerator with scaling sub-matrix partition. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 1032–1047. IEEE, 2024.
- [39] Jiezhong Qiu, Hao Ma, Omer Levy, Scott Wen-tau Yih, Sinong Wang, and Jie Tang. Blockwise self-attention for long document understanding. *arXiv preprint arXiv:1911.02972*, 2019.
- [40] Zheng Qu, Liu Liu, Fengbin Tu, Zhaodong Chen, Yufei Ding, and Yuan Xie. Dota: detect and omit weak attentions for scalable transformer acceleration. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 14–26, 2022.
- [41] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.
- [42] P Rajpurkar. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250*, 2016.
- [43] Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy Lillicrap, Jean-baptiste Alayrac, Radu Soriccut, Angeliki Lazariidou, Orhan Firat, Julian Schrittwieser, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- [44] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021.
- [45] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642, 2013.
- [46] Quan Sun, Yufeng Cui, Xiaosong Zhang, Fan Zhang, Qiyang Yu, Yueze Wang, Yongming Rao, Jingjing Liu, Tiejun Huang, and Xinlong Wang. Generative multimodal models are in-context learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14398–14409, 2024.
- [47] Yi Tay, Dara Bahri, Liu Yang, Donald Metzler, and Da-Cheng Juan. Sparse sinkhorn attention. In *International Conference on Machine Learning*, pages 9438–9447. PMLR, 2020.
- [48] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- [49] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. Glue: a multi-task benchmark and analysis platform for natural language understanding. corr abs/1804.07461 (2018). *arXiv preprint arXiv:1804.07461*, 2018.
- [50] Hanrui Wang, Zhekai Zhang, and Song Han. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 97–110. IEEE, 2021.
- [51] Yiqi Wang, Wentao Chen, Xiaotian Han, Xudong Lin, Haiteng Zhao, Yongfei Liu, Bohan Zhai, Jianbo Yuan, Quanzeng You, and Hongxia Yang. Exploring the reasoning abilities of multimodal large language models (mllms): A comprehensive survey on emerging trends in multimodal reasoning. *arXiv preprint arXiv:2401.06805*, 2024.
- [52] A Warstadt. Neural network acceptability judgments. *arXiv preprint arXiv:1805.12471*, 2019.
- [53] Adina Williams, Nikita Nangia, and Samuel R Bowman. A broad-coverage challenge corpus for sentence understanding through inference. *arXiv preprint arXiv:1704.05426*, 2017.
- [54] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, pages 38–45, 2020.
- [55] Qizhe Xie, Guokun Lai, Zihang Dai, and Eduard Hovy. Large-scale cloze test dataset created by teachers. *arXiv preprint arXiv:1711.03225*, 2017.
- [56] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- [57] Amir Yazdanbakhsh, Ashkan Moradifrouzabadi, Zheng Li, and Mingyu Kang. Sparse attention acceleration with synergistic in-memory pruning and on-chip recomputation. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 744–762. IEEE, 2022.
- [58] Haoran You, Zhanyi Sun, Huihong Shi, Zhongzhi Yu, Yang Zhao, Yongan Zhang, Chaojian Li, Baopu Li, and Yingyan Lin. Vitcod: Vision transformer acceleration via dedicated algorithm and accelerator co-design. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 273–286. IEEE, 2023.
- [59] Wenhao Yu, Chenguang Zhu, Zaitang Li, Zhiting Hu, Qingyun Wang, Heng Ji, and Meng Jiang. A survey of knowledge-enhanced text generation. *ACM Computing Surveys*, 54(11s):1–38, 2022.
- [60] Pengchuan Zhang, Xiyang Dai, Jianwei Yang, Bin Xiao, Lu Yuan, Lei Zhang, and Jianfeng Gao. Multi-scale vision longformer: A new vision transformer for high-resolution image encoding. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 2998–3008, 2021.
- [61] Jieru Zhao, Pai Zeng, Guan Shen, Quan Chen, and Minyi Guo. Hardware-software co-design enabling static and dynamic sparse attention mechanisms. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [62] Zhe Zhou, Junlin Liu, Zhenyu Gu, and Guangyu Sun. Energon: Toward efficient acceleration of transformers using dynamic sparse attention. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(1):136–149, 2022.